

Documentacion Tecnica delCodigo - Sprint 2

Futsi Mini ERP | Actualizado 2026-05-28

1. Arquitectura

El repositorio esta dividido en back/ para Django REST Framework y front/ para React, Vite y Tailwind. La base local puede correr SQLite; produccion queda preparada para Supabase Postgres. El frontend se publica como estatico y consume el API por VITE_API_URL.

- Backend: Django, DRF, modelos core, serializers, viewsets, endpoints de reportes, facturas y reconocimiento facial.
- Frontend: React en un solo entry principal, estilos Tailwind/CSS, roles, dashboards y flujos de caja/familia/coach/contador.
- Documentos: generados desde tools/build_sprint2_updated_documents.py para mantener consistencia.

2. Endpoints agregados o reforzados en Sprint 2

Endpoint	Metodo	Uso
/api/reports/accounting.xlsx	GET	Descarga Excel contable valido para admin/owner/accounting.
/api/invoices/	GET	Lista facturas segun rol.
/api/invoices/simulate/	POST	Genera factura simulada de cargo o gasto.
/api/invoices/{id}/pdf/	GET	Descarga PDF simulado.
/api/invoices/{id}/xml/	GET	Descarga XML simulado.
/api/face-attendance/recognize/	GET	Diagnostico de motor DeepFace/mock.
/api/face-attendance/recognize/	POST	Compara foto capturada contra roster y registra asistencia.
/api/historical-imports/	GET	Lista cargas historicas para admin/contador.
/api/historical-imports/preview/	POST	Carga Excel, password opcional y genera preview editable.
/api/historical-imports/{id}/commit/	POST	Firma importacion y crea pagos/gastos historicos.

3. Archivos clave modificados

Archivo	Responsabilidad
back/core/models.py	Invoice y FaceRecognitionAttempt; relaciones con cargos, pagos, gastos, alumnos y sesiones.
back/core/views.py	Excel contable, PAC simulado, PDF/XML, diagnostico y reconocimiento DeepFace.
back/core/serializers.py	Serializers para facturas e intentos faciales.
back/futsi_api/settings.py	Config Postgres/Supabase por variables separadas y proteccion ante URL malformada.
front/src/main.tsx	UI de facturas, Excel, PWA, camara, recuadro facial, perfiles y roles.
front/src/styles.css	Responsive mobile, espejo de camara, estados de reconocimiento y tema oscuro.
front/public/*	Manifest, icono y service worker PWA.
front/android/*	Proyecto Android Capacitor y APK debug para emulador/dispositivo.

Archivo	Responsabilidad
render.yaml / Dockerfile	Preparacion de deploy Render con migraciones y variables.

4. Ejecucion local recomendada

- Backend normal: cd back; python -m venv .venv; .\venv\Scripts\python.exe -m pip install -r requirements.txt; .\venv\Scripts\python.exe manage.py migrate; .\venv\Scripts\python.exe manage.py seed_demo --reset; .\venv\Scripts\python.exe manage.py runserver.
- Backend con DeepFace local: crear .venv en raiz e instalar requirements + deepface + tf-keras; correr manage.py con --noreload para evitar doble carga.
- Frontend: cd front; npm install; npm run dev.
- Validacion: manage.py test, npm run build, abrir http://127.0.0.1:5173 y revisar roles admin/contador/coach/cajero/familia.
- Android: cd front; npm run build:android; npx cap sync android; abrir front/android en Android Studio o usar tools/build_android_debug_apk.ps1.

5. Notas de DeepFace

- DeepFace queda opcional y local porque TensorFlow/DeepFace pesan demasiado para una demo gratis en Render.
- El backend compara la captura normal y espejada para compensar camaras frontales.
- Si DeepFace esta instalado pero no hay match confiable, el sistema reporta sin coincidencia; ya no inventa asistencia por fallback.
- FACE_MATCH_MAX_DISTANCE permite ajustar umbral; default 0.55 para demo.

6. Importacion historica Excel implementada

- Modelos HistoricalImport y HistoricalImportRow guardan archivo original, usuario que subio, usuario que firma, password usado, resumen, filas y destino creado.
- El endpoint preview carga Excel con openpyxl y msoffcrypto-tool cuando hay cifrado, detecta hojas INGRESOS SEDES/GASTOS SEDES y genera placeholders editables.
- El endpoint commit exige firma responsable, respeta filas omitidas, crea Payment para ingresos y Expense para egresos, y registra AuditLog.
- La ampliacion Sprint 3 debe sumar staging completo para ARCHIVO DE OPERACION, ESTADO DE RDOS. y demas hojas historicas.

7. Seguridad y QA tecnica para Sprint 3

Area	Accion Sprint 3	Criterio de salida
SQL injection	Revisar queries crudas, filtros, reportes y parsers; mantener ORM parametrizado y pruebas con payloads maliciosos.	Suite automatizada confirma que filtros/login/importacion no ejecutan SQL no esperado.
Autenticacion	Validar sesiones/token, expiracion, logout, cambio de password y bloqueo de fuerza bruta.	Casos negativos cubiertos y logs de intentos fallidos.
Autorizacion por rol	Pruebas de acceso horizontal/vertical para admin, contador, cajero, coach, tutor y coordinador.	Cada endpoint sensible tiene test de permisos.

Area	Accion Sprint 3	Criterio de salida
Carga de archivos	Validar extension, MIME, tamano, password, hojas permitidas y limpieza de Excel historico.	Archivos corruptos/enormes/protegidos fallan de forma controlada.
Datos personales	Revisar fotos, datos medicos, responsivas y facturas; minimizar exposicion por rol.	Cajero/coach/tutor solo ven datos necesarios.
CORS/CSRF	Endurecer origenes permitidos, headers, HTTPS y cookies si se cambia estrategia de auth.	Produccion no acepta origenes comodin.
Secretos	Validar Render/GitHub/Supabase sin secretos en repo.	git-secrets/trufflehog o equivalente sin hallazgos criticos.
Dependencias	Ejecutar npm audit/pip-audit y fijar actualizaciones criticas.	Sin vulnerabilidades altas conocidas o con excepcion documentada.

Herramienta	Uso propuesto	Resultado esperado
SonarQube / SonarCloud	Analisis estatico de frontend/backend en GitHub Actions: bugs, smells, cobertura, duplicacion y hotspots.	Quality Gate visible por PR/main.
pytest + Django	Tests unitarios/API para modelos, permisos, pagos, facturas, importacion Excel, reportes y auditoria.	Suite backend reproducible en CI.
Playwright Python	Pruebas e2e usando elementos reales de la web: login por rol, navegacion movil, pagos, asistencia, Excel historico y facturas.	Flujos criticos pasan en Chrome headless.
Pruebas agresivas UI	Scripts que llenan formularios con valores largos, caracteres raros, fechas invalidas, montos negativos, clicks rapidos y reloads.	La app no se rompe visualmente ni guarda datos invalidos.
OWASP ZAP baseline	Escaneo automatizado contra entorno staging local/Render.	Reporte sin riesgos altos antes de piloto.
Lighthouse	Revisar PWA, accesibilidad y performance movil.	PWA instalable y sin problemas severos de accesibilidad.
Android smoke	Instalar APK debug en emulador y validar login, menu, tema oscuro y camara.	APK usable antes de compartir demo.

6. Contrato de Excel contable

Hoja	Contenido
Resumen	Ingresos confirmados, egresos aprobados, utilidad, gastos pendientes, cargos abiertos por sede.
Pagos	Pago individual con cliente, concepto, metodo, canal, estado, monto, fechas y receptor.
Cargos	Adeudos programados por alumno/equipo, concepto, monto, vencimiento y estado.
Gastos	Gasto por sede, categoria, proveedor, monto, estado, capturo/aprobo.
Descuentos	Solicitudes y aprobaciones de descuento con motivo y usuario.
Asistencia con adeudo	Registros donde un alumno asistio teniendo saldo abierto.
Facturas	UUID, tipo, receptor, concepto, subtotal, impuesto, total y fecha.

7. Contrato de facturacion simulada

- source_type=expense genera factura de egreso ligada a Expense.
- source_type=charge genera factura de ingreso ligada a Charge, Student y Guardian cuando aplica.
- El UUID se genera con uuid4 y no representa timbrado fiscal real.
- El XML se guarda como texto en DB; el PDF se guarda como archivo generado por reportlab.
- Los roles guardian y coach solo ven facturas asociadas a ellos; cajero ve facturas de su sede.

8. Pipeline facial

- 1 Frontend abre camara y muestra video espejo.
- 2 Al presionar Pasar lista se captura un frame en canvas.
- 3 Frontend envia session, image base64 y opcional student forzado.
- 4 Backend obtiene roster por sede/grupo y filtra alumnos activos/prueba/pausa/lesion.
- 5 Backend compara contra fotos de alumnos con DeepFace y tambien prueba imagen espejada.
- 6 Si distancia <= FACE_MATCH_MAX_DISTANCE registra asistencia presente y bitacora.
- 7 Si no hay match, devuelve Sin coincidencia y no inventa asistencia.

9. Variables de entorno relevantes

Variable	Uso
DJANGO_SECRET_KEY	Clave secreta backend.
DJANGO_DEBUG	false en produccion.
DJANGO_ALLOWED_HOSTS	Hosts Render/dominio.
CORS_ALLOWED_ORIGINS	Origenes Pages/local permitidos.
DB_ENGINE	postgres para Supabase.
POSTGRES_DB/USER/PASSWORD/HOST/PORT/SSLMODE	Conexion Supabase separada para evitar errores de URL.
FACE_MATCH_MAX_DISTANCE	Umbral de reconocimiento facial local.
VITE_API_URL	URL del backend para React.
VITE_BASE_PATH	Base path Pages /futsi/.

10. Diseno tecnico de importacion historica Sprint 3

- Crear comando Django `import\_historical\_excel` o endpoint admin protegido para subir archivos historicos.
- Leer hojas con parser estructurado, no con manipulacion de strings; aceptar multiples layouts por hoja.
- Guardar archivo original y metadatos de carga en `historical\_import\_batches`.
- Guardar filas crudas normalizadas en staging antes de crear Payment, Expense, Charge o metricas historicas.
- Generar reporte de errores descargable: fila, hoja, columna, valor, razon y sugerencia.
- Crear llaves naturales para deduplicar: periodo, sede, folio, clave, concepto, monto, fecha y fuente.
- Exponer dashboard de conciliacion: total origen vs total importado vs diferencia por hoja/sede/mes.